SOC Network Traffic Investigation & Detection Lab

Network Traffic Analysis, Detection & SIEM Investigation

Prepared by:
Shardul Deshpande

Tools: Nmap • Wireshark • Splunk Cloud • Kali Linux • Ubuntu • VirtualBox

1.Objectives

The primary objectives of this project were to:

- Simulate basic network reconnaissance and HTTP activity in a controlled virtual lab environment.
- Capture and analyze network traffic using Wireshark.
- Identify open and closed TCP ports using Nmap.
- Analyze TCP connection behavior, including SYN, SYN-ACK, and RST packets.
- Investigate HTTP requests and identify potentially suspicious URI patterns.
- Identify indicators associated with potential command-injection activity.
- Ingest structured network events into Splunk Cloud.
- Create SPL queries to search for suspicious network activity.
- Develop basic detection logic for suspicious HTTP requests.
- Build a Splunk dashboard to visualize network activity and detected events.
- Document the investigation process and findings from a SOC analyst perspective.

2.Lab Environment

The investigation was conducted in an isolated virtual lab environment using VirtualBox.

Virtual Machines:

- **Kali Linux** — used as the testing and reconnaissance machine.
- **Ubuntu Linux** — used as the target/server machine.

Network:

- VirtualBox host-only network
- Kali IP: `192.168.56.101`
- Ubuntu IP: `192.168.56.102`

Tools Used:

- **Nmap** — network reconnaissance and port scanning
- **Wireshark** — packet capture and traffic analysis
- **Python SimpleHTTPServer** — used to generate HTTP traffic in the lab
- **Splunk Cloud** — event ingestion, searching, detection, and visualization

- **VirtualBox** — virtual lab infrastructure

The environment was intentionally isolated to ensure that all reconnaissance and suspicious HTTP activity remained within the controlled lab network.

3.Traffic & Attack Simulation

Network activity was generated within the isolated lab environment to simulate common SOC investigation scenarios.

The activity consisted of:

1. **Network reconnaissance** using Nmap to identify reachable ports and services on the Ubuntu machine.
2. **TCP connection analysis** using Wireshark to observe connection attempts and responses.
3. **HTTP traffic generation** using a Python SimpleHTTPServer running on port `8000`.
4. Several HTTP requests were generated to represent different levels of interest:
    - `GET /` — normal web request
    - `GET /admin` — potential administrative path probing
    - `GET /?cmd=whoami` — suspicious request containing a command-related parameter

All activity was performed against the Ubuntu VM from the Kali VM and remained inside the controlled VirtualBox network.

---

4. Nmap Reconnaissance & Findings

Nmap was used from the Kali Linux VM to perform reconnaissance against the Ubuntu target at `192.168.56.102`.

The initial scan showed that the host was reachable and responded to ARP requests. During the TCP scan, the majority of the scanned ports were reported as closed or in ignored states.

A more focused scan was then performed against selected ports:

Port	State	Observation
22/tcp	Closed	Connection attempts were reset
80/tcp	Closed	Connection attempts were reset
443/tcp	Closed	Connection attempts were reset
8000/tcp	Open	HTTP service detected

Port **8000/tcp** was the main service identified during reconnaissance. It was associated with an HTTP service provided by the Python SimpleHTTPServer used in the lab.

The Nmap results were subsequently correlated with Wireshark packet captures to examine how the target responded at the TCP level.

Key finding: Port `8000/tcp` was the only confirmed open port among the specifically tested ports, providing the HTTP service that was investigated in the next phase.

---

5. Wireshark Packet Analysis

Wireshark was used to capture and analyze the network traffic generated during the Nmap scans and HTTP activity.

The TCP packet captures provided visibility into how the Kali machine communicated with the Ubuntu target.

TCP Connection Behavior

For the open HTTP service on port `8000`, the expected TCP handshake was observed:

1. **Kali → Ubuntu:** SYN
2. **Ubuntu → Kali:** SYN-ACK

3. **Kali → Ubuntu:** connection completion/reset behavior was observed depending on the scan and connection method.

For closed ports such as `80` and `443`, connection attempts resulted in TCP **RST** responses, indicating that the target was not accepting connections on those ports.

Packet Capture Observations

Wireshark was also used to:

- Inspect TCP flags and connection behavior.
- Filter traffic by TCP port.
- Identify HTTP packets within the capture.
- Follow HTTP streams to reconstruct the communication.
- Examine individual HTTP request details.
- Correlate Nmap results with the underlying packet-level activity.

The packet captures provided the raw network evidence used for the later HTTP investigation and Splunk analysis.

---

6. HTTP Traffic Investigation

After identifying port `8000/tcp` as an open HTTP service, the traffic was examined in Wireshark to understand the requests being sent to the Ubuntu server.

The Python HTTP server generated normal web responses that could be inspected through captured HTTP packets and followed streams.

Several HTTP requests were observed during the investigation:

- `GET /` — normal request to the server root.
- `GET /admin` — request for an administrative-looking path.
- `GET /?cmd=whoami` — request containing a `cmd` parameter.

The `/?cmd=whoami` request was treated as suspicious because command-related parameters can be associated with command-injection attempts.

However, the presence of the parameter alone does **not** prove that command execution occurred. The investigation therefore treated it as a **potential command-injection indicator**, rather than confirmed successful exploitation.

Wireshark's HTTP stream functionality was used to correlate the request and server response and provide packet-level evidence for the subsequent Splunk investigation.

---

7. Splunk Cloud Investigation & Event Ingestion

Splunk Cloud was used as the SIEM platform for the investigation. Structured network events from the lab were exported into a CSV file and uploaded to Splunk Cloud for analysis.

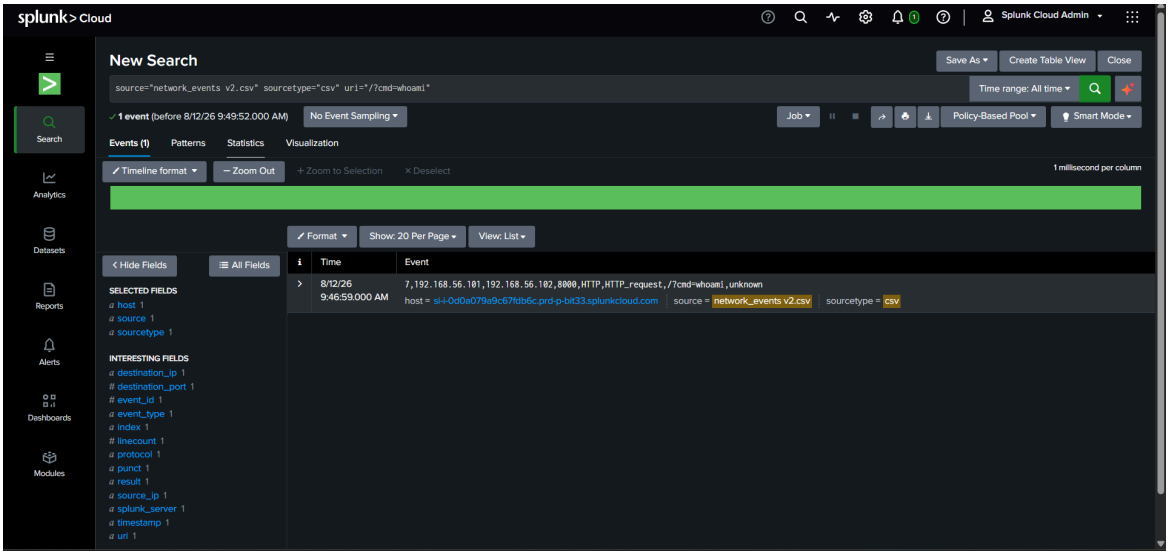
The dataset contained information such as:

- Source and destination IP addresses
- Destination ports
- Protocol
- Event type
- HTTP URI
- Result/status

After ingestion, the events were searched using Splunk Search Processing Language (SPL).

The investigation confirmed that the uploaded dataset contained **7 events**, including Nmap reconnaissance activity and HTTP requests.

Figure 1: Splunk Cloud search showing the captured `/?cmd=whoami` HTTP event.



Basic searches were used to:

- Review all imported network events.
- Identify HTTP requests.
- Locate requests containing the `cmd` parameter.
- Count activity by source IP.
- Count activity by destination port.

The Splunk analysis provided a centralized view of the network activity and allowed the packet-level findings from Wireshark to be represented as searchable security events.

---

8. Detection Rules & Suspicious Activity

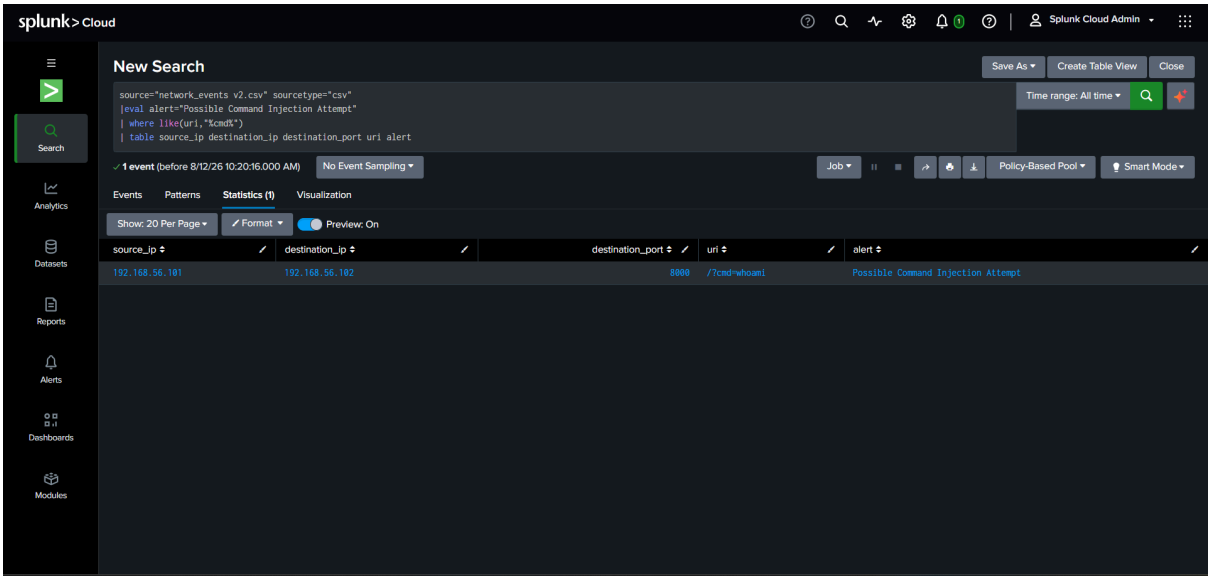
SPL detection logic was created to identify HTTP requests containing potentially suspicious URI patterns.

Detection 1 — Command Injection Indicator

The following search identifies URIs containing the `cmd` parameter:

```
source="network_events v2.csv" sourcetype="csv"
| where like(uri,"%cmd%")
| eval alert="Possible Command Injection Attempt"
| table source_ip destination_ip destination_port uri alert
```

Figure 2: SPL detection identifying the `/?cmd=whoami` request as a Possible Command Injection Attempt



This detection identified:

`/?cmd=whoami`

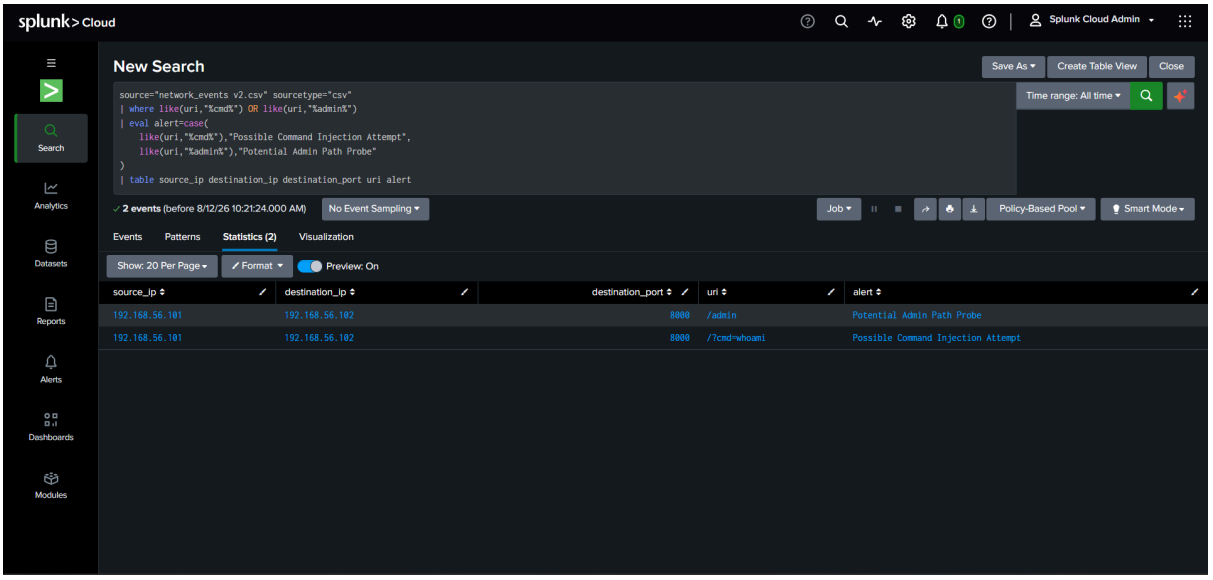
The event was classified as a **Possible Command Injection Attempt**.

Detection 2 — Administrative Path Probing

A second detection searched for both command-related and administrative URI patterns:

```
source="network_events v2.csv" sourcetype="csv"
| where like(uri,"%cmd%") OR like(uri,"%admin%")
| eval alert=case(
    like(uri,"%cmd%"),"Possible Command Injection Attempt",
    like(uri,"%admin%"),"Potential Admin Path Probe"
)
| table source_ip destination_ip destination_port uri alert
```

Figure 3: Combined detection identifying potential administrative path probing and command-injection activity.



The detection identified two events:

- `/admin` → **Potential Admin Path Probe**
- `/?cmd=whoami` → **Possible Command Injection Attempt**

These detections demonstrate how raw network activity can be converted into simple, searchable SOC detection logic.

---

9. Splunk SOC Dashboard

A Splunk dashboard was created to provide a visual summary of the network activity observed during the investigation.

The dashboard contains four panels:

1. Network Activity by Destination Port

Displays the number of events associated with each destination port. Port `8000` showed the primary activity because it hosted the HTTP service used in the lab.

2. HTTP Requests by URI

Displays the observed HTTP request paths, including `/`, `/admin`, and `/?cmd=whoami`.

3. Suspicious HTTP Activity

Visualizes the events identified by the detection logic:

- Possible Command Injection Attempt
- Potential Admin Path Probe

4. Activity by Source IP

Shows the number of events generated by each source IP. The Kali machine (`192.168.56.101`) generated the observed activity against the Ubuntu target.

The dashboard provides a centralized view of reconnaissance and HTTP activity and demonstrates how a SOC analyst can move from raw network data to visualized security findings.

Figure 4: Splunk Cloud SOC dashboard summarizing network and suspicious HTTP activity.

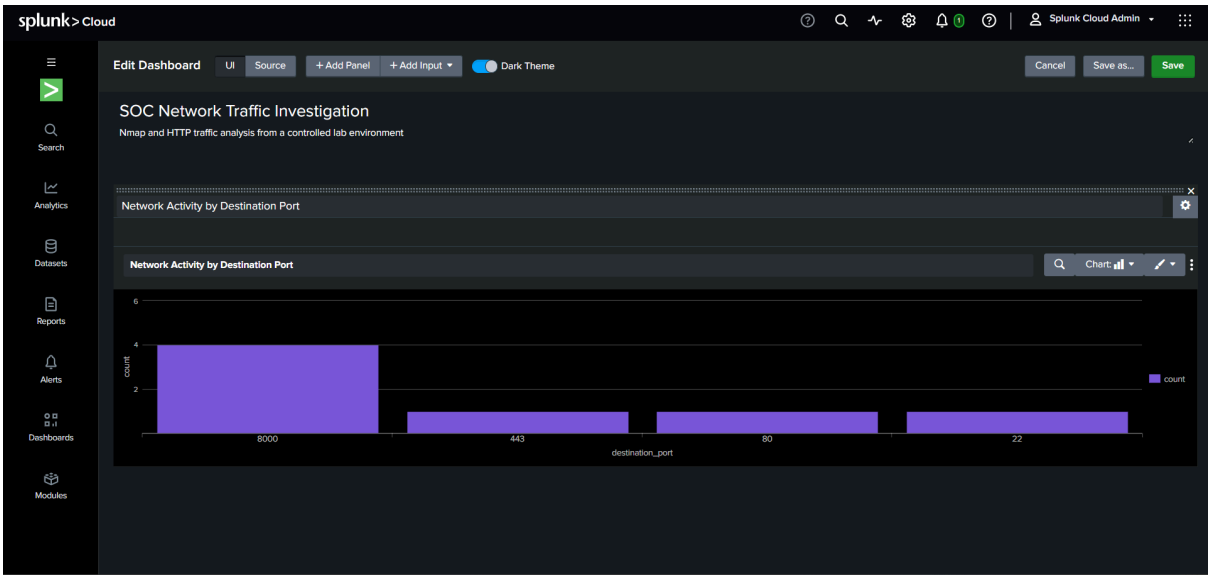


Figure 5: Additional Splunk dashboard visualization of the investigated network events and detections.

---

10. Investigation Findings & Analysis

The investigation established a clear sequence of activity from reconnaissance to HTTP-based suspicious behavior.

Finding 1 — Network reconnaissance

The Ubuntu host at `192.168.56.102` was reachable from Kali. Focused scanning identified port `8000/tcp` as open, while ports 22, 80, and 443 were closed.

Finding 2 — HTTP service

The open port `8000` hosted an HTTP service. Wireshark confirmed HTTP communication between the Kali and Ubuntu systems.

Finding 3 — Administrative path probing

The request:

```
GET /admin
```

was identified as potentially suspicious because it targeted an administrative-looking path. It was classified as a **Potential Admin Path Probe**.

Finding 4 — Command-related URI

The request:

`GET /?cmd=whoami`

contained a command-related parameter and was therefore flagged as a **Possible Command Injection Attempt**.

Finding 5 — No confirmed command execution

Although the request contained `cmd=whoami`, the available packet evidence did not establish that the command was actually executed on the Ubuntu host. Therefore, the activity is classified as a **potential attack attempt**, not confirmed successful exploitation.

Overall assessment

The investigation successfully demonstrated the SOC workflow of identifying network activity, collecting packet-level evidence, extracting relevant events, creating detections, and visualizing the results in a SIEM.

---

11. Limitations & Lessons Learned

Limitations

- The investigation was performed in a controlled virtual lab and does not represent a production enterprise network.
- The dataset ingested into Splunk was manually structured from the observed activity rather than collected from a production log source.
- The `/?cmd=whoami` request was identified as suspicious, but successful command execution could not be confirmed from the available evidence.
- The investigation used a small number of events, so the detections were designed primarily to demonstrate the investigation workflow rather than provide production-ready detection coverage.

Lessons Learned

This project provided practical experience with the SOC investigation lifecycle, including:

- Network reconnaissance with Nmap.
- TCP and HTTP packet analysis with Wireshark.
- Following HTTP streams to understand network communication.
- Identifying suspicious indicators in HTTP requests.
- Writing SPL searches in Splunk Cloud.
- Creating basic detection logic.
- Building a SIEM dashboard to visualize security events.
- Correlating network-level evidence with SIEM data.
- Distinguishing between a suspicious indicator and confirmed exploitation.

---

12. Conclusion

This project demonstrated a complete, controlled SOC investigation workflow, beginning with network reconnaissance and progressing through packet analysis, HTTP investigation, SIEM-based detection, and security visualization.

Nmap was used to identify exposed services, while Wireshark provided packet-level visibility into TCP and HTTP communication. The investigation identified an open HTTP service on port `8000` and highlighted potentially suspicious requests targeting `/admin` and `/?cmd=whoami`.

The relevant network events were then ingested into Splunk Cloud, where SPL searches and detection logic were developed to identify suspicious HTTP activity. A dashboard was also created to provide a centralized view of network activity and potential security indicators.

Although command execution could not be confirmed, the project demonstrated the ability to identify, investigate, document, and visualize suspicious activity using tools commonly associated with SOC operations.

Overall, the project strengthened practical skills in network traffic analysis, SIEM investigation, detection engineering, and security event correlation.

---

13. Tools & Skills Demonstrated

Tools Used

- **Kali Linux** — reconnaissance and network testing
- **Ubuntu Linux** — target/server environment
- **VirtualBox** — isolated virtual lab environment
- **Nmap** — network discovery and port/service reconnaissance
- **Wireshark** — packet capture, TCP analysis, HTTP inspection, and stream following
- **Python SimpleHTTPServer** — controlled HTTP traffic generation
- **Splunk Cloud** — SIEM-based event searching, detection, and visualization
- **SPL (Search Processing Language)** — security event analysis and detection logic

Skills Demonstrated

- Network reconnaissance
- TCP/IP traffic analysis
- HTTP traffic analysis
- Packet capture and filtering
- Identification of open and closed services
- Basic security indicator analysis
- SIEM data ingestion
- SPL querying
- Detection rule creation
- Security event correlation
- SOC dashboard development
- Incident investigation and documentation
- Evidence-based security analysis
